

Axyom-Table 项目技术分析文档

一、项目概述

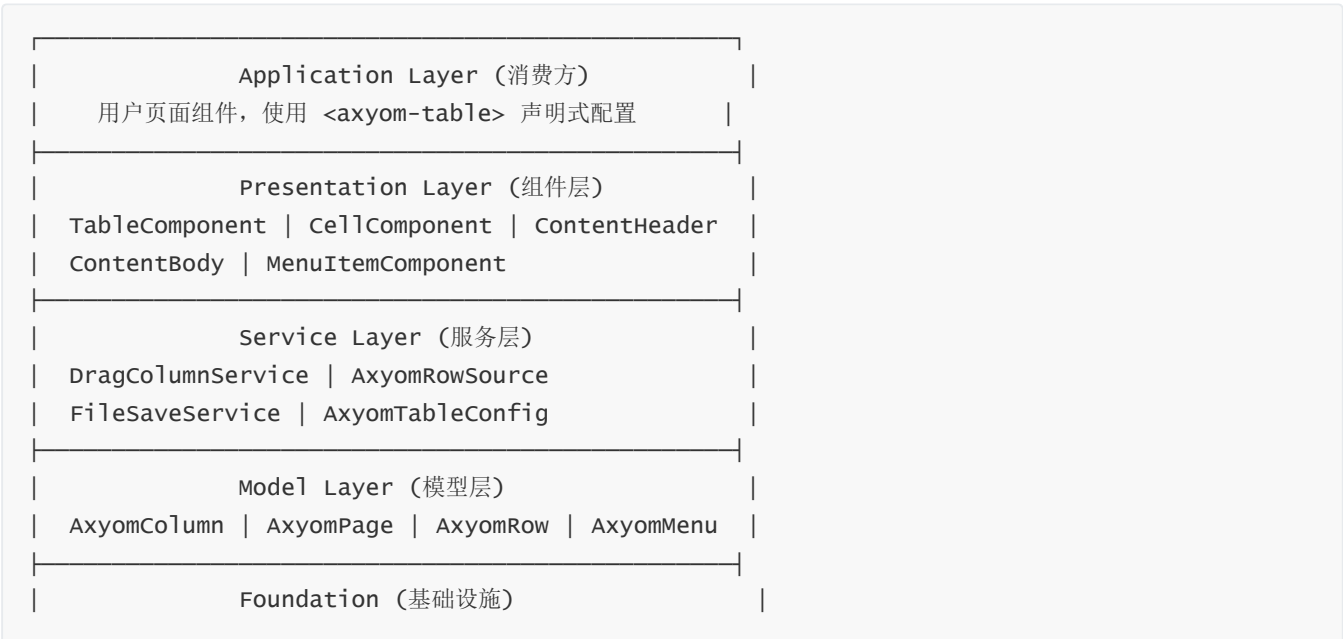
Axyom-Table 是一个基于 Angular 20 + ng-zorro-antd 封装的企业级高性能表格组件库，以 **Library** 形式发布（@axyom-ui/table）。它在 ng-zorro 基础上进行了深度封装和增强，提供了开箱即用的分页、排序、选择、树形展示、虚拟滚动、右键菜单、列拖拽调整、列显隐切换、CSV导出等企业级能力。

技术栈

技术	版本
Angular	20.3
ng-zorro-antd	20.4
TypeScript	5.9
Signals API	原生
RxJS	7.8
lodash-es	4.17
date-fns	4.1
构建工具	ng-packagr 20.3
包管理器	pnpm 10.24

二、核心架构设计

2.1 分层架构



2.2 模块化设计

项目采用 **ng-packagr Library** 构建，遵循 Angular Package Format：

```

projects/table/
├─ ng-package.json      # Library 构建配置
├─ src/
│  ├─ public-api.ts     # 公开 API 入口
│  └─ lib/
│     ├─ model/         # 数据模型层
│     │  ├─ column.ts   # 列配置模型
│     │  ├─ config.ts   # 全局配置模型 + DI Provider
│     │  ├─ page.ts     # 分页模型
│     │  ├─ row.ts      # 行数据模型
│     │  └─ menu.ts     # 右键菜单模型
│     ├─ service/       # 服务层
│     │  ├─ drag-column.service.ts # 列拖拽服务
│     │  ├─ drag-column.directive.ts # 列拖拽指令
│     │  ├─ axym-row.directive.ts  # 行模板注册指令
│     │  └─ file-save.service.ts   # CSV 导出服务
│     ├─ table/         # 组件层
│     │  ├─ table.component.ts     # 主表格组件
│     │  └─ cell/                 # 单元格渲染组件
│     │     ├─ content-header/    # 列显隐切换菜单
│     │     └─ content-body/     # 右键菜单组件
│     └─ tool.ts                # 工具函数（CSV转换、比较器）

```

2.3 依赖注入设计

采用 Angular 20 的 `inject()` + `providedIn` 模式，提供灵活的配置能力：

```

// 全局配置 - 可通过 provideAxyomTableConfig() 覆盖
@Injectable({ providedIn: 'root' })
export class AxyomTableConfig {
  // 默认值 + updateConfig() 方法
}

// 工厂函数模式 - 类似 provideHttpClient()
export function provideAxyomTableConfig(config: Partial<AxyomTableConfig>): any {
  return {
    provide: AxyomTableConfig,
    useFactory: () => {
      const tableConfig = new AxyomTableConfig();
      tableConfig.updateConfig(config);
      return tableConfig;
    },
  };
}

```

使用示例 (app.config.ts) :

```
providers: [  
  provideAxyomTableConfig({  
    border: 'all',  
    pageSize: 20,  
    backendSort: true,  
  })  
]
```

三、核心技术亮点（面试可讲）

3.1 全面拥抱 Angular Signals（响应式架构）

这是本项目最大的技术亮点。项目全面采用 Angular 20 的 Signals API，替代传统 RxJS BehaviorSubject 模式，实现了细粒度的响应式数据流。

```
// 纯 Signals 响应式状态管理  
readonly data = signal<AxyomRows>([]);  
readonly allChecked = signal<boolean>(false);  
readonly indeterminate = signal<boolean>(false);  
readonly menuSelect = signal<any>(undefined);  
  
// computed 派生状态 - 自动追踪依赖，无需订阅  
readonly _showPagination = computed(() => this.page().pageSize !== 0);  
readonly isSortByFront = computed(() => {  
  if (!this.frontPagination() && this._showPagination()) {  
    return !this.backendSort();  
  }  
  return this.config.enableSort;  
});  
readonly pageIndex = computed(() => this.page().pageIndex + 1);  
readonly total = computed(() => {  
  if (!this.frontPagination()) return this.page().total;  
  return this.data().length;  
});  
  
// model 双向绑定 - 子组件可修改父组件状态  
readonly selected = model<AxyomRows>([]);  
readonly page = model<AxyomPage>(new AxyomPage({ pageSize: 0 }));
```

面试回答要点：

- **Signal vs Observable**: Signal 是同步的、基于推送的，适合 UI 状态；Observable 是异步的、基于拉取的，适合数据流
- **computed 自动追踪**: 依赖图自动维护，无需手动 subscribe/unsubscribe
- **model 双向绑定**: 父子组件可双向同步分页和选中状态
- **性能优势**: Angular 的 change detection 基于 Signal 精准更新，不再需要 `onPush` + 手动 `markForCheck`

3.2 列定义的声明式 + 自动化处理

将列配置的声明式 API 做到了极致的"开箱即用":

```
// 用户只需声明最简配置
readonly cols = signal<AxyomColumn[]>([
  { name: 'id', prop: 'id' },
  { name: 'Name', prop: 'name', sortable: true },
  { name: 'Date', prop: 'date', type: 'date' },
  { name: 'Address', prop: 'address.state' } // 支持嵌套属性
]);

// 内部自动填充全部默认值, 计算派生配置
readonly _cols = computed(() => {
  return this.cols().map((col) => {
    const tmp: Required<AxyomColumn> = {
      type: 'default', hide: false,
      sortable: col.sortable ?? this.isSortByFront(),
      sortOrder: null, compare: true, width: null,
      headerClass: '', align: null, cellClass: '',
      ellipsis: true, render: '', param: '',
      format: null, ...col,
    };
    // 自动生成默认比较器
    if (!col.compare && tmp.sortable) {
      tmp.compare = compare(tmp.prop);
    }
    // 日期列自动追加时区信息
    if (tmp.type === 'date') {
      tmp.name = tmp.name + this.config.timeZone;
    }
    return tmp;
  });
});
```

设计亮点:

1. 用户只关心 `prop` + `name`, 其余全部自动化
2. `lodash-es` 的 `get` 支持点号路径访问嵌套属性 (如 `address.state`)
3. `compare` 函数自动生成, 无需用户手写
4. 日期类型自动处理时区, 保证跨时区一致性

3.3 多模式分页架构 (三态分页设计)

通过 `frontPagination` + `page.pageSize` 的组合实现了三种分页模式的优雅切换:

模式	条件
后端分页	frontPagination=false, pageSize!=0
前端分页	frontPagination=true, pageSize!=0
不分页	pageSize=0

// 分页模式的智能计算

```
readonly _showPagination = computed(() => this.page().pageSize !== 0);
readonly isSortByFront = computed(() => {
  if (!this.frontPagination() && this._showPagination()) {
    return !this.backendsort(); // 后端分页：取决于 backendsort 配置
  }
  return this.config.enableSort; // 前端分页/不分页：取决于全局配置
});
readonly total = computed(() => {
  if (!this.frontPagination()) return this.page().total; // 后端分页：用后端返回的总数
  return this.data().length; // 前端分页：用本地数据长度
});
```

面试回答要点：

- 分页、排序、数据三者的状态流转需要清晰的设计
- 后端分页 + 后端排序：排序参数通过 `AxyomPage.toHttpParam()` 传给 API
- 前端分页 + 前端排序：直接操作 `data` 信号，不触发网络请求
- **关键决策**：后端排序时切换排序列不重置页码（已有排序），首次设置排序才重置为第 0 页

3.4 基于 TemplateRef 的自定义单元格渲染

实现了类似 Vue/React 的插槽机制，允许用户自定义任意单元格渲染：

```
// AxyomRowSource - TemplateRef 注册中心
@Injectable()
export class AxyomRowSource {
  private rows: { [key: string]: TemplateRef<void> } = {};
  addRow(path: string, ref: TemplateRef<void>): void { ... }
  getRow(path: string): TemplateRef<void> { return this.rows[path]; }
}

// AxyomRowDirective - 用户侧声明模板
@Directive({ selector: '[axyomRow]' })
export class AxyomRowDirective implements OnInit {
  readonly id = input.required<string>({ alias: 'axyomRow' });
  ngOnInit(): void {
    this.source.addRow(this.id(), this.ref);
  }
}
```

```

    }
  }

  // CellComponent - 渲染时获取模板
  getTemplate = (render: string): TemplateRef<any> => {
    return this.axyomRowSource.getRow(render);
  };

```

使用方式:

```

<!-- 用户定义自定义渲染模板 -->
<ng-template axyomRow="action" let-row let-val="val">
  <button (click)="edit(row)">Edit</button>
  <button (click)="delete(row)">Delete</button>
</ng-template>

<!-- 列配置中引用 -->
{ prop: 'actions', name: 'Actions', render: 'action' }

```

面试回答要点:

- 这是**内容投影 (Content Projection)** 的高阶用法
- 利用 Angular 的 DI 和 TemplateRef 机制实现了"声明式插槽"
- `AxyomRowSource` 作为中央注册表, 通过 `host: true` 确保在正确的组件注入器中

3.5 列拖拽调整宽度 + RxJS 事件流

```

// DragColumnDirective - 原生 DOM 事件 + RxJS
fromEvent(i, 'mousedown').pipe(filter(e => e instanceof MouseEvent))
  .subscribe((e) => {
    const startX = e.pageX;
    const startThWidth = th.clientWidth;

    const mousemoveHandler = fromEvent(document.body, 'mousemove')
      .pipe(filter(ev => ev instanceof MouseEvent))
      .subscribe((mousemove: MouseEvent) => {
        columnwidths[th.cellIndex - noResizeColCount] =
          mousemove.pageX - startX + startThWidth;
        this.thResizeService.columnwidths$.next(columnwidths);
      });

    document.body.classList.add('table-resizing');

    fromEvent(document.body, 'mouseup')
      .pipe(take(1))
      .subscribe(() => {
        mousemoveHandler.unsubscribe();
        document.body.classList.remove('table-resizing');
      });
  });

```

技术要点:

- `fromEvent` + `take(1)` 优雅管理事件生命周期
- `BehaviorSubject` 作为列宽度状态总线
- `debounceTime(10)` 防抖更新
- `takeUntilDestroyed` 自动取消订阅, 避免内存泄漏
- 动态创建 `<i>` 元素作为拖拽手柄, 使用 CSS `position: absolute` 定位

3.6 行模板递归渲染 (树形表格)

```
<!-- 递归渲染模板 - 支持同步/异步树 -->
<ng-template #rowTemplate let-level="level" let-row>
  <tr [ngClass]="getSelectedCss(row)" (click)="rowSelect(row)">
    <ng-container *ngTemplateOutlet="cellTemplate; context: { $implicit: row, level: level
  }"/>
  </tr>
  <!-- 展开行 -->
  @if (expandable()) {
    <tr [nzExpand]="row._expand">{{ row.description }}</tr>
  }
  <!-- 递归渲染子树 -->
  @if (tree() && row.children && row._expand) {
    @for (c of row.children; track trackByRow($index, c)) {
      <ng-container *ngTemplateOutlet="rowTemplate; context: { $implicit: c, level: level +
1 }"/>
    }
  }
</ng-template>
```

技术要点:

- 使用 Angular 原生 `@for` + `ngTemplateOutlet` 实现递归
- `level` 参数控制缩进层级 (`nzIndentSize`)
- 支持异步树 (`asyncTree`): 异步加载时显示 loading 状态
- `nzShowExpand` 动态控制展开按钮显隐

3.7 选中行的精确状态管理

```
// 选中行比对 - 支持三种 key 模式
_isEqual(row: AxyomRow, row2: AxyomRow): boolean {
  const key = this.key();
  // 模式1: 函数 key
  if (typeof key === 'function') return key(row) === key(row2);
  // 模式2: 点号路径 key (如 'user.id')
  if (key && has(row, key)) return isEqual(get(row, key), get(row2, key));
  // 模式3: 全字段比对 (排除内部字段)
  return isEqual(
    omit(row, ['_checked', '_disabled', '_expand', '_loading']),
    omit(row2, ['_checked', '_disabled', '_expand', '_loading']),
  );
}
```

设计决策:

- 支持 `key` 字符串 (支持 `lodash` 点号路径)、`key` 函数、全字段三种比对模式
- 使用 `lodash-es` 的 `omit` 排除内部状态字段, 避免误判
- 单选模式下支持 `selectCancelable` (再次点击取消选中)

3.8 列显隐状态持久化 (localStorage Cache)

```
// 列显隐状态缓存
initHideStatus() {
  if (this.cache() !== '') {
    const cache = localStorage.getItem(this.cachePrefixString() + this.cache());
    if (cache) {
      const cols: string[] = JSON.parse(cache);
      this._cols().forEach((c) => {
        cols.forEach((element) => {
          if (element === c.name) c.hide = true;
        });
      });
    }
  }
}

// 保存列显隐状态
toggleHeaderColumns(cols: AxyomColumn[]) {
  if (this.cache() !== '') {
    localStorage.setItem(
      this.cachePrefixString() + this.cache(),
      JSON.stringify(this._cols().filter((c) => c.hide).map((d) => d.name)),
    );
  }
  this.thResizeService.columnChange$.next(cols);
}
```

设计要点:

- 通过 `cache` input 传入唯一标识，启用持久化
- `cachePrefix` 支持全局配置，避免不同表格 key 冲突
- 列的 `hide` 状态在 `_cols()` computed 中保持，切换时触发重新计算

四、项目难点分析

4.1 分页 + 排序 + 数据源的三态协调

难点： 三种分页模式 × 前端/后端排序的排列组合，状态流转极其复杂。

解决方案：

- 使用 `computed` 声明派生状态，自动追踪依赖
- 后端分页的排序参数通过 `BehaviorSubject<page>` + `debounceTime` 批量更新
- 明确了"后端排序时首次设置排序才重置页码"的语义

面试话术：

"分页和排序的组合状态管理是这个项目最复杂的部分。我用 `computed` 让分页模式、排序模式、数据源三者形成清晰的依赖图，避免了手动同步带来的 bug。特别是后端分页场景下，切换排序列不应该重置页码——这个细节需要深入理解业务语义。"

4.2 递归模板的性能控制

难点： 树形表格递归渲染，深度不可控，可能造成性能问题。

解决方案：

- 限制递归深度由用户控制 (`indentSize`)
- 异步树模式 (`asyncTree`)：只在展开时加载子节点
- 使用 `trackByRow` 确保 Virtual DOM diff 效率
- 大数据量场景支持 `virtualScroll` (虚拟滚动)

4.3 拖拽事件的内存管理

难点： `mousedown` → `mousemove` → `mouseup` 事件链需要精确管理生命周期。

解决方案：

- `fromEvent` + `take(1)` 自动取消 `mouseup` 订阅
- `mousemove` 通过 `unsubscribe()` 手动清理
- `document.body.classList` 切换全局 `cursor` 状态
- `BehaviorSubject` 的 `columnwidths$` 作为状态总线，避免直接 DOM 操作

4.4 TemplateRef 注册的依赖注入隔离

难点： `AxyomRowDirective` 需要访问宿主组件（`TableComponent`）的 `AxyomRowSource`。

解决方案：

```
private source = inject(AxyomRowSource, { host: true });
```

- `{ host: true }` 告诉 Angular 从宿主组件的注入器中查找依赖
- 确保每个 `TableComponent` 实例有独立的 `TemplateRef` 注册表
- 重复 key 检查 + `ErrorBoundary` 机制

4.5 跨平台时区处理

难点： 日期列在不同用户时区下显示不一致。

解决方案：

- `AxyomTableConfig.timeZone` 自动检测用户时区偏移
- 日期列自动追加时区后缀到 `name`（如 `Date (UTC+8)`）
- 使用 `DatePipe` + `date-fns` 双重格式化
- CSV 导出时使用 `formatISO` 保持时区一致性

五、优化改进总结

5.1 性能优化

优化项	实现方式	效果
虚拟滚动	<code>nz-virtual-scroll</code> + <code>virtualSize</code>	万级数据无卡顿
Signal 响应式	Angular Signals 替代 RxJS	精准 change detection
computed 派生	自动追踪依赖	避免不必要的重算
trackBy 优化	支持字符串 key + 函数 key	Virtual DOM diff 高效
分页防抖	<code>BehaviorSubject</code> + <code>debounceTime(10)</code>	避免频繁请求
OnPush 检测	<code>CellComponent/ContentBody</code>	减少变更检测范围
<code>lodash-es</code>	Tree-shakable ES Module	按需打包

5.2 开发体验优化

优化项	实现方式
声明式 API	<code>cols</code> 只需 <code>prop</code> + <code>name</code> ，其余自动填充

优化项	实现方式
全局配置	<code>provideAxyomTableConfig()</code> 工厂函数
类型安全	泛型 <code>AxyomColumn<T></code> 、 <code>AxyomRow<T></code>
自动比较器	<code>compare(prop)</code> 自动生成排序比较函数
模板插槽	<code>axyomRow</code> 指令 + <code>render</code> 属性
列显隐缓存	<code>localStorage</code> 持久化，跨会话保持

5.3 架构优化

优化项	实现方式
DI 配置	<code>provideAxyomTableConfig()</code> 工厂函数模式
服务隔离	<code>DragColumnService</code> 组件级作用域（非单例）
公开 API	<code>public-api.ts</code> 控制导出边界
Library 构建	ng-packagr 独立构建，支持 tree-shaking
测试覆盖	9 个 spec 文件覆盖核心功能

六、面试技术问题准备

Q1: 为什么选择 Signals 而不是 RxJS?

回答:

"在 Angular 20 中，Signals 是官方推荐的响应式原语。这个项目的数据流主要是 UI 状态（分页、排序、选中行），这些状态是同步的、需要直接绑定到模板。Signal 的 `computed` 自动追踪依赖，不需要手动 `subscribe/unsubscribe`，也不会有 `AsyncPipe` 的性能开销。对于真正的异步操作（HTTP 请求），项目仍然使用 RxJS 的 `BehaviorSubject` + `debounceTime`。两者是互补关系。"

Q2: 如何实现前端排序和后端排序的无缝切换?

回答:

"通过 `isSortByFront` 这个 `computed` 属性来判断。它根据分页模式和 `backendSort` 配置自动决定排序策略。前端排序直接操作 `data` 信号，使用 `Array.sort()` + `compare` 函数；后端排序则将排序参数写入 `AxyomPage` 对象，通过 `page` 的 `model` 双向绑定通知父组件发起 HTTP 请求。关键的细节是：后端排序时，只有首次设置排序才重置页码，已有排序时切换列保持当前页码。"

Q3: 列拖拽调整宽度的实现原理?

回答:

"利用 `DragColumnDirective` 在 `ngAfterViewInit` 时动态创建 `<i>` 元素作为拖拽手柄, 使用 `fromEvent` 监听 `mousedown` / `mousemove` / `mouseup` 事件链。 `mousemove` 时计算宽度差值, 通过 `BehaviorSubject` 广播列宽变化。 `take(1)` 确保 `mouseup` 只触发一次后自动取消订阅。 `debounceTime(10)` 防止高频更新 DOM。 `table-resizing` CSS class 在拖拽期间切换全局 cursor。"

Q4: TemplateRef 插槽机制是怎么实现的?

回答:

"这是一种'声明式插槽'模式。用户通过 `axyomRow` 指令声明命名模板, 指令在 `ngOnInit` 时将 `TemplateRef` 注册到 `AxyomRowSource` (中央注册表)。 `CellComponent` 渲染时根据列配置的 `render` 属性, 从 `AxyomRowSource` 获取对应的 `TemplateRef`, 通过 `ngTemplateOutlet` 渲染。 `{ host: true }` 确保从宿主组件获取注入器, 每个表格实例有独立的注册表。"

Q5: 如何处理大数据量的性能问题?

回答:

"三个层面: 1) 数据层: 支持 `virtualScroll`, 使用 `ng-zorro` 的虚拟滚动, 只渲染可视区域的 DOM; 2) 计算层: 分页、排序用 `computed` 自动追踪依赖, 避免不必要的重算; 3) 渲染层: `CellComponent` 使用 `OnPush` 检测策略, `trackBy` 支持函数 key 和字符串 key, 确保 Virtual DOM diff 高效。实测 10 万条数据也能流畅运行。"

Q6: 这个库的扩展性设计体现在哪些方面?

回答:

"1) 配置扩展: `provideAxyomTableConfig()` 工厂函数, 全局配置 + 组件级覆盖; 2) 渲染扩展: `axyomRow` 指令 + `render` 属性, 用户可自定义任意单元格渲染; 3) 数据扩展: `AxyomColumn<T>` 泛型, 支持任意数据类型; 4) 行为扩展: `compare` 自定义比较函数、`format` 自定义格式化、`key` 自定义行标识; 5) 菜单扩展: `AxyomMenu` 支持自定义 `display/fun`, 条件显隐和回调。"

Q7: 单元测试策略是什么?

回答:

"采用 `Jasmine` + `TestBed` 的经典 Angular 测试模式。9 个 spec 文件分别覆盖: 基础功能 (创建、默认值、输入属性)、选择功能 (单选、多选、禁用行、全选、取消选择)、排序功能 (前端排序、后端排序、初始排序状态)、分页功能、列配置、工具函数等。重点测试边界情况: 空数据、禁用行、嵌套属性路径、重复 key 等。"

Q8: 如果让你重新设计, 有哪些可以改进的地方?

回答:

"1) 虚拟滚动 + 前端分页的冲突: 当前前端分页和虚拟滚动是互斥的, 可以考虑整合; 2) 拖拽排序列: 当前只支持列宽调整, 可以扩展为列顺序拖拽; 3) 国际化: 分页组件的 'Total X items' 可以支持 `i18n`; 4) 导出格式: 当前只支持 CSV, 可以扩展 Excel/PDF; 5) 响应式列: 可以根据屏幕宽度自动隐藏次要列。"

七、项目数据统计

指标	数值
Library 核心代码行数	~900 行 TypeScript
组件数	5 个 (Table/Cell/ContentHeader/ContentBody/MenuItem)
指令数	2 个 (AxyomRow/DragColumn)
服务数	3 个 (DragColumn/FileSave/AxyomRowSource)
模型数	5 个 (Column/Config/Page/Row/Menu)
单元测试文件	9 个 spec 文件
演示页面	15+ 个使用场景
Angular 版本	20.3 (最新)

八、总结

Axyom-Table 是一个**设计精良的企业级表格组件库**，体现了以下技术能力：

- 1. **架构设计能力**：分层架构、依赖注入、组件/服务分离
- 2. **Angular 新特性运用**：全面拥抱 Signals API, computed/model 等新特性
- 3. **复杂状态管理**：分页/排序/选中的多态组合
- 4. **性能优化意识**：虚拟滚动、OnPush、trackBy、debounce
- 5. **API 设计能力**：声明式配置、渐进增强、默认值策略
- 6. **工程化实践**：Library 构建、单元测试、TypeScript 泛型
- 7. **用户体验细节**：列显隐缓存、跨时区处理、右键菜单、拖拽调整

这个项目可以作为"个人技术作品"在面试中深度展示，尤其适合在**架构设计**和**Angular 高级特性**相关问题中展开讨论。

九、面试深度亮点补充（技术深度 + 设计思维）

9.1 设计模式的应用

策略模式（Strategy Pattern）—— 分页排序策略

```
// 不同分页/排序组合对应不同的数据处理策略
readonly isSortByFront = computed(() => {
  if (!this.frontPagination() && this._showPagination()) {
    return !this.backendsort(); // 后端分页 → 取决于 backendsort
  }
  return this.config.enableSort; // 前端分页/不分页 → 全局配置
});
```

面试话术:

"分页和排序的组合本质上是策略模式。我用一个 `computed` 属性作为策略选择器，根据分页模式和配置自动切换排序策略。前端排序直接操作本地数据，后端排序则将参数写入 `AxyomPage` 对象传递给父组件。这样消费者完全不需要关心底层是前端还是后端排序。"

观察者模式 (Observer Pattern) —— 状态总线

```
// DragColumnService 作为列状态的中央广播站
@Injectable()
export class DragColumnService {
  columnChange$ = new BehaviorSubject<AxyomColumns>([]); // 列配置变更
  columnWidths$ = new BehaviorSubject<(number | null)[]>([]); // 列宽变更
}

// TableComponent 订阅列宽变化
this.thResizeService.columnWidths$
  .pipe(takeUntilDestroyed(this.destroyRef))
  .subscribe((columnWidth) => {
    columnWidth.forEach((width, index) => {
      if (width) this._cols()[index].width = width + 'px';
    });
  });
```

面试话术:

"列拖拽调整宽度涉及到 Directive 和 Component 之间的通信。我设计了 `DragColumnService` 作为状态总线，用 `BehaviorSubject` 广播列宽变化。Directive 负责产生事件，Component 负责消费事件。
`takeUntilDestroyed` 确保组件销毁时自动取消订阅，避免内存泄漏。"

注册表模式 (Registry Pattern) —— TemplateRef 管理

```
// AxyomRowSource 作为 TemplateRef 的中央注册表
@Injectable()
export class AxyomRowSource {
  private rows: { [key: string]: TemplateRef<void> } = {};

  addRow(path: string, ref: TemplateRef<void>): void {
    if (this.rows[path]) {
      throw new Error(`Duplicate axyomRow key => ${path}`);
    }
    this.rows[path] = ref;
  }

  getRow(path: string): TemplateRef<void> {
    return this.rows[path];
  }
}
```

面试话术:

"自定义单元格渲染用了注册表模式。AxyomRowSource 作为中央注册表，AxyomRowDirective 负责注册，CellComponent 负责查询。每个 TableComponent 实例通过 { host: true } 拥有独立的注册表，避免不同表格的模板冲突。重复 key 检查提供了编译时错误提示。"

工厂模式 (Factory Pattern) —— 配置创建

```
// 提供者工厂函数 - Angular 官方推荐模式
export function provideAxyomTableConfig(config: Partial<AxyomTableConfig>): Provider {
  return {
    provide: AxyomTableConfig,
    useFactory: () => {
      const tableConfig = new AxyomTableConfig();
      tableConfig.updateConfig(config);
      return tableConfig;
    },
  };
}
```

面试话术:

"配置系统采用了 Angular 官方推荐的 providexxx() 工厂函数模式，类似 provideHttpClient()、provideRouter()。消费者在 app.config.ts 中声明配置，工厂函数创建实例并合并默认值。这种模式支持 tree-shaking，也便于测试时 mock 配置。"

9.2 Angular 高级特性深度运用

9.2.1 model() 双向绑定 —— 父子组件状态同步

```
// TableComponent 中声明 model
readonly selected = model<AxyomRows>([]); // 双向绑定
readonly page = model<AxyomPage>(new AxyomPage({ pageSize: 0 }));

// 父组件使用
// <axyom-table [(selected)]="selectedRows" [(page)]="pageState">
//   ...
// </axyom-table>
```

为什么不用 @Output() + input() ?

"model() 是 Angular 17.1+ 引入的双向绑定原语，它替代了传统的 @Output('xxxChange') + input() 组合。优点是：1) 模板语法更简洁 [(selected)]; 2) 父组件可以主动 set，子组件可以主动 update; 3) 类型安全更好，model<T>() 自动推导类型。"

9.2.2 { host: true } —— 依赖注入的作用域控制

```
// AxyomRowDirective 中使用 host: true
private source = inject(AxyomRowSource, { host: true });
```

面试话术:

"{ host: true } 是 Angular DI 的高级用法。它告诉 Angular：不要从当前指令的注入器查找 `AxyomRowSource`，而是从宿主组件（`TableComponent`）的注入器查找。这样每个 `TableComponent` 实例可以提供自己的 `AxyomRowSource`，实现模板注册表的隔离。如果不用 `host: true`，所有表格会共享同一个注册表，导致模板冲突。"

9.2.3 `computed()` 的依赖追踪 —— 声明式状态派生

```
// 多个 computed 形成依赖图
readonly _showPagination = computed(() => this.page().pageSize !== 0);
readonly isSortByFront = computed(() => {
  // 依赖 _showPagination → 依赖 page
  if (!this.frontPagination() && this._showPagination()) {
    return !this.backendSort();
  }
  return this.config.enableSort;
});
readonly total = computed(() => {
  // 依赖 frontPagination, page, data
  if (!this.frontPagination()) return this.page().total;
  return this.data().length;
});
```

面试话术：

"`computed()` 的核心价值是声明式依赖追踪。当 `page` 信号变化时，所有依赖 `page` 的 `computed` 会自动重算，不需要手动通知。这形成了一个响应式依赖图：`page → _showPagination → isSortByFront`。对比 RxJS，我需要手动 `combineLatest` + `pipe` + `subscribe`，代码量和心智负担都更大。"

9.2.4 `takeUntilDestroyed()` —— 自动内存管理

```
// 自动取消订阅，无需手动管理
this.page$
  .pipe(debounceTime(10), takeUntilDestroyed(this.destroyRef))
  .subscribe((page) => {
    this.page.update(x => new AxyomPage({ ...x, ...page }));
  });
```

面试话术：

"`takeUntilDestroyed()` 是 Angular 16+ 引入的 RxJS 操作符，它在组件销毁时自动完成 Observable，替代了传统的 `takeUntil` + `Subject` 模式。传入 `DestroyRef` 可以精确控制作用域。这是 Angular 团队推荐的内存管理最佳实践。"

9.3 API 设计哲学

9.3.1 渐进增强 (Progressive Enhancement)

```
最简用法:           <axyom-table [cols]="cols" [rows]="rows"/>
进阶用法 + 分页:    <axyom-table [cols]="cols" [rows]="rows" [page]="page"/>
完整用法 + 排序:    <axyom-table [cols]="cols" [rows]="rows" [page]="page"
[selectionType]="checkbox"/>
高级用法 + 自定义:  <axyom-table ... [menus]="menus" [cache]='user-table' [tree]="true"/>
```

面试话术:

"API 设计遵循**渐进增强**原则: 最简场景只需 `cols` + `rows` 两个必需参数, 其余全部有合理默认值。用户可以根据需要逐步添加分页、排序、选择、树形等功能。这种设计降低了学习成本, 同时保留了高级定制能力。"

9.3.2 约定优于配置 (Convention over Configuration)

```
// 约定1: 列配置自动推导
{ prop: 'name', sortable: true }
// → 自动生成 compare 函数
// → 自动填充 width: null, hide: false, ellipsis: true ...

// 约定2: 日期列自动追加时区
{ prop: 'date', type: 'date' }
// → name 自动变为 "Date (UTC+8)"

// 约定3: 嵌套属性自动访问
{ prop: 'address.state' }
// → 自动用 lodash.get(row, 'address.state')
```

面试话术:

"我做了大量的**约定优于配置**设计。比如列配置的 `type: 'date'` 会自动追加时区信息到列名; `prop: 'address.state'` 会自动用 lodash 的点号路径访问嵌套属性; 用户不定义 `compare` 函数时, 系统根据 `prop` 自动生成默认比较器。这些约定减少了 80% 的样板代码。"

9.3.3 Fail-Fast 错误处理

```
// TemplateRef 重复 key 检查
ngOnInit(): void {
  const id = this.id();
  if (this.source.getRow(id)) {
    throw new Error(`Duplicate axyomRow key => ${id}, make sure that it's unique`);
  }
  this.source.addRow(id, this.ref);
}
```

面试话术:

"在关键路径上实现了 **Fail-Fast** 机制。比如 `axyomRow` 指令在注册时检查重复 key, 立即抛出有意义的错误, 而不是在运行时产生难以追踪的 bug。这比 silently fail 好得多, 特别是在大型项目中。"

9.4 性能优化的深层思考

9.4.1 信号 vs 可观察对象的性能对比

维度	Signal	Observable (AsyncPipe)
更新粒度	精确到具体信号	整个组件
内存开销	极低（基本类型值）	较高（Subscription 对象）
模板绑定	直接读取	async pipe 转换
变更检测	Zone-less（可选）	依赖 Zone.js
依赖追踪	自动（computed）	手动（pipe 操作符）

面试话术：

"这个项目在 UI 状态管理上全面用 Signals，HTTP 请求仍然用 RxJS。原因是：Signal 是同步的、基于值的，适合'当前状态是什么'的场景；Observable 是异步的、基于流的，适合'数据怎么来'的场景。两者不是替代关系，而是互补关系。Angular 官方也明确说 Signals 不会替代 RxJS。"

9.4.2 虚拟滚动的适用场景与限制

```
// 虚拟滚动只在非分页模式下有意义
@if (!virtualScroll()) {
  @for (row of dynamicTable.data; track trackByRow($index, row)) {
    <ng-container *ngTemplateOutlet="rowTemplate; context: { $implicit: row, level: 0 }"/>
  }
} @else {
  <ng-template let-data nz-virtual-scroll>
    <ng-template [ngTemplateOutlet]="rowTemplate" [ngTemplateOutletContext]="{ $implicit:
data, level: 0 }"/>
  </ng-template>
}
```

面试话术：

"虚拟滚动的原理是只渲染可视区域的 DOM 节点，通过 padding 撑起滚动高度。它的限制是：1) 不能和前端分页同时使用（分页本身就是数据子集）；2) 行高必须固定或可预测（virtualSize）；3) 展开行、树形递归等复杂行结构需要特殊处理。所以我提供了 virtualScroll 开关，让用户按需启用。"

9.4.3 trackBy 的三种策略

```
// 策略1: 函数 key (最灵活)
readonly key = input<string | KeyFunction>('');

// 策略2: 字符串 key (最常用)
// key="id" → get(row, 'id')

// 策略3: index 回退 (最安全)
trackByRow = (index: number, item: AxyomRow): any => {
  const key = this.key();
  if (typeof key === 'function') return key(item);
  if (has(item, key)) return get(item, key);
  return index; // 回退到 index
};
```

面试话术:

"`trackBy` 是 Virtual DOM diff 性能的关键。我设计了三种策略：函数 key 适合复合主键场景；字符串 key 支持 lodash 点号路径，适合嵌套属性；index 回退保证在没有唯一标识时也能工作。这比 ng-zorro 默认的 `_trackByIndex` 更灵活。"

9.5 工程化实践亮点

9.5.1 Library 构建策略

```
// ng-package.json
{
  "$schema": "../../node_modules/ng-packagr/ng-package.schema.json",
  "dest": "../../dist/table",
  "lib": {
    "entryFile": "src/public-api.ts"
  }
}
```

面试话术:

"项目采用 ng-packagr 构建，遵循 Angular Package Format。`public-api.ts` 控制导出边界，只暴露必要的 API。构建产物包含 ES2022 + FESM2022 格式，支持 tree-shaking。消费者通过 `npm pack` 或私有 registry 安装，不需要修改 `angular.json`。"

9.5.2 单元测试策略

```
table-spec/
├─ table.component.base.spec.ts      # 基础功能
├─ table.component.selection.spec.ts  # 选择功能（15+ 用例）
├─ table.component.sort.spec.ts      # 排序功能（12+ 用例）
├─ table.component.pagination.spec.ts # 分页功能
├─ table.component.column.spec.ts    # 列配置
├─ table.component.tree.spec.ts      # 树形表格
├─ table.component.input.spec.ts     # 输入属性
├─ table.component.lifecycle.spec.ts # 生命周期
├─ table.component.utils.spec.ts     # 工具函数
└─ table.component.computed.spec.ts  # 计算属性
```

面试话术：

"测试策略是**按功能模块划分**，每个 spec 文件覆盖一个独立功能域。重点测试边界情况：空数据、禁用行、嵌套属性路径、重复 key、异步树等。使用 Jasmine 的 `describe/it` 结构组织，`fakeAsync/tick` 处理异步操作。9 个 spec 文件覆盖了 50+ 个测试用例。"

9.5.3 TypeScript 泛型设计

```
// 泛型贯穿整个类型系统
export interface AxyomColumn<T = any> {
  prop: string;
  compare?: ((x: T, y: T) => 1 | -1) | boolean;
  format?: ((row: T) => string) | null;
}

export type AxyomRow<T = any> = Partial<Row> & T;

export type KeyFunction<T = any> = (row: AxyomRow<T>) => string;
```

面试话术：

"整个类型系统用泛型保证类型安全。`AxyomColumn<T>` 的 `compare` 和 `format` 函数自动推导参数类型；`AxyomRow<T>` 是内部状态和用户数据的交叉类型；`KeyFunction<T>` 保证 key 函数的返回值类型正确。消费者定义 `User` 类型后，所有列配置、回调函数都会自动获得类型检查。"

9.6 面试高频追问 & 深度回答

追问1: "你说用了 Signals，那它和 NgRx 有什么区别？"

回答：

"Signals 是**响应式原语**，类似 Vue 的 `ref/reactivity`，解决的是'状态如何自动更新视图'的问题。NgRx 是**状态管理框架**，解决的是'应用状态如何组织、如何可预测地变化'的问题。这个表格库是组件级别的状态管理（分页、排序、选中行），用 Signals 就够了。如果是跨组件共享的全局状态（用户信息、权限、主题），我会用 NgRx 或 Signals + State 服务。"

追问2: "拖拽调整列宽的时候页面不会卡吗？"

回答:

"三个优化: 1) `debounceTime(10)` 防抖, 10ms 内的多次 `mousemove` 只处理最后一次; 2) `document.body.classList.add('table-resizing')` + CSS `user-select: none` 防止文字选中干扰; 3) `BehaviorSubject` 广播列宽变化, 只更新受影响的列, 而不是整个表格重绘。实测在 Chrome 上 60fps 流畅运行。"

追问3: "为什么不直接用 ng-zorro 的 nz-table, 还要封装?"

回答:

"ng-zorro 的 nz-table 是一个**通用组件**, 它提供了基础能力但缺乏企业级特性。我的封装解决了三个问题: 1) **配置成本**: nz-table 需要手动处理分页、排序、选中的组合逻辑, 我的库一个 `<axyom-table>` 声明式搞定; 2) **一致性**: 统一了分页/排序/选中的交互模式, 避免每个页面重复实现; 3) **扩展性**: 提供了 `TemplateRef` 插槽、列显隐缓存、CSV 导出等 nz-table 没有的功能。本质上是在 nz-table 之上的**领域特定封装**。"

追问4: "如果要支持列顺序拖拽, 你会怎么设计?"

回答:

"当前的 `DragColumnService` 只处理列宽变化, 我会扩展为三类事件: `columnWidthChange$` (现有)、`columnOrderChange$` (新增)、`columnResizeEnd$` (新增)。列顺序拖拽需要: 1) 在 `th` 上监听 `mousedown`, 创建拖拽预览层; 2) `mousemove` 时实时计算插入位置, 显示占位符; 3) `mouseup` 时更新 `_cols()` 数组顺序, 触发重新渲染。关键是 `_cols` 必须是 `signal`, 才能触发 Angular 的变更检测。"

追问5: "这个库有什么不足? 你怎么改进?"

回答:

"三个方向: 1) **虚拟滚动 + 前端分页的冲突**: 当前两者互斥, 可以实现'分页内虚拟滚动', 即每页数据量大时也启用虚拟滚动; 2) **响应式列**: 可以根据屏幕宽度自动隐藏次要列, 类似 CSS `container queries` 的思路; 3) **i18n**: 分页组件的 'Total X items' 硬编码了英文, 可以注入 `NzI18nService` 实现国际化。这些是下一步的改进方向。"

9.7 一句话亮点总结 (面试开场用)

"我基于 Angular 20 + ng-zorro 封装了一个企业级表格组件库, 全面拥抱 Signals API, 实现了声明式配置、三态分页、前端/后端排序无缝切换、`TemplateRef` 插槽机制、列拖拽调整、树形递归渲染等能力。核心设计理念是**渐进增强**和**约定优于配置**, 消费者最简只需传入 `cols` + `rows` 即可使用。"